

SIMATS ENGINEERING

Assessment Tool 1 – Answer Key

Course: Database Management Systems (CSA05) | CO5: Query Processing, Optimization & Transaction Management

Activity Tool: Constraint-Based Problem Solving (High)

Q1. Query execution plan for a query with nested joins & sub-queries — apply heuristic optimization rules step-by-step. [5 Marks]

Sample Query:

```
SELECT E.ename
FROM Employee E, Department D
WHERE E.deptno = D.deptno
      AND D.dname = 'Sales'
      AND E.salary > (SELECT AVG(salary) FROM Employee)
```

Step 1 — Initial (Canonical) Parse Tree

The parser converts the SQL into a relational-algebra tree with a Cartesian product for every table in the FROM clause and a single SELECT (σ) node on top carrying every WHERE-clause condition, plus the sub-query evaluated separately.

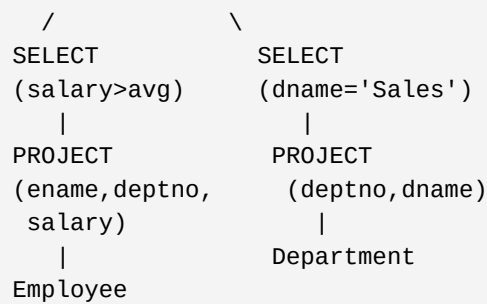
```
PROJECT (ename)
  |
  SELECT (E.deptno=D.deptno AND D.dname='Sales' AND E.salary > AVG_sal)
    |
    X (Cartesian Product)
  /  \
Employee(E)  Department(D)
```

Step 2 — Apply Heuristic Rewrite Rules

- **Rule 1 - Break up conjunctive selections:** split the single SELECT into a cascade of three SELECT nodes, one per condition, so each can be optimized independently.
- **Rule 2 - Push SELECT operations down the tree:** move $\sigma(D.dname='Sales')$ next to Department, and evaluate the sub-query $AVG(salary)$ once and push $\sigma(E.salary>avg)$ next to Employee, before the join.
- **Rule 3 - Push PROJECT operations down (project early):** project Employee to (ename, deptno, salary) and Department to (deptno, dname) immediately after the base relations, discarding unused columns as early as possible.
- **Rule 4 - Replace Cartesian Product + Selection with a JOIN:** combine the X node and the $\sigma(E.deptno=D.deptno)$ node into a single equi-join, which the DBMS can implement via hash-join or sort-merge instead of a full product.
- **Rule 5 - Order joins so the operation producing the smallest intermediate result runs first:** since $\sigma(dname='Sales')$ on Department is highly selective (few rows), evaluate it before joining, drastically reducing the join's outer input.

Step 3 — Optimized Execution Plan

```
PROJECT (ename)
  |
  JOIN (E.deptno = D.deptno)
```



This heuristically optimized tree filters and projects each relation before the join, so the join operator processes far fewer tuples than the naive Cartesian-product plan — reducing both I/O and CPU cost.

Q2. Apply cost-based optimization to choose between nested-loop, sort-merge and hash join. [5 Marks]

Given data: Relation R has 10,000 tuples stored in 100 disk blocks (100 tuples/block); Relation S has 5,000 tuples stored in 50 disk blocks (100 tuples/block). Available buffer pages $M = 12$ (10 usable for build/partition + 2 reserved).

Relation	Tuples	Blocks (B)	Tuples/Block
R	10,000	100	100
S	5,000	50	100

(a) Block Nested-Loop Join — $\text{cost} = B(R) + \lceil B(R)/(M-2) \rceil \times B(S)$

Using the smaller relation S as the outer loop and R as inner: with $M-2 = 10$ buffer blocks available for the outer relation, $\lceil 50/10 \rceil = 5$ outer passes are needed.

Cost = $B(S) + \lceil B(S)/(M-2) \rceil \times B(R) = 50 + (5 \times 100) = 550$ block transfers.

(b) Sort-Merge Join — $\text{cost} = \text{sort}(R) + \text{sort}(S) + (B(R)+B(S))$

Assuming both relations already fit the two-pass external sort with $M=12$ buffers (runs of size $M=12$ blocks, so $\lceil \log_{M-1}(B/M) \rceil$ merge passes):

Sort cost for R $\approx 2 \times B(R) \times (1 + \lceil \log_{10}(100/12) \rceil) \approx 2 \times 100 \times 2 = 400$; Sort cost for S $\approx 2 \times 50 \times 2 = 200$.

Merge/join phase = $B(R)+B(S) = 150$.

Total cost $\approx 400 + 200 + 150 = 750$ block transfers (worst case; if inputs are already sorted on the join attribute this drops to just 150).

(c) Hash Join — $\text{cost} = 3(B(R)+B(S))$, when the smaller relation's partitions fit in memory

With $M=12$ buffers we can build $M-1 = 11$ partitions. Partitioning step reads+writes both relations once ($2 \times (B(R)+B(S)) = 300$); the probe step reads both partitioned relations once more ($B(R)+B(S) = 150$).

Total cost = $3 \times (100+50) = 450$ block transfers, provided each partition of S ($\approx 50/11 \approx 5$ blocks) fits comfortably in the remaining memory buffers.

Decision

Comparing the three totals — Hash Join (450) < Block Nested-Loop (550) < Sort-Merge (750) — the **query optimizer selects the Hash Join**, since it gives the lowest estimated I/O cost given the available memory. Sort-Merge would be preferred instead only if the output needed to be produced in sorted order (e.g. followed by ORDER BY or GROUP BY on the join key), and Nested-Loop is preferable only when a usable index exists on the inner relation's join attribute (Index Nested-Loop Join).

Q3. JDBC connectivity to MySQL — parameterized queries using PreparedStatement. [5 Marks]

Steps: (1) load/register the driver, (2) open a Connection to the MySQL server, (3) create a PreparedStatement with placeholders (?), (4) bind parameters and execute, (5) process the ResultSet, (6) close all resources (preferably with try-with-resources to auto-close on exit).

```
import java.sql.*;

public class EmployeeDAO {
    private static final String URL =
        "jdbc:mysql://localhost:3306/company_db?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASS = "password";

    public static void main(String[] args) {
        String sql = "SELECT ename, salary FROM Employee " +
            "WHERE deptno = ? AND salary > ?";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement pst = con.prepareStatement(sql)) {

            pst.setInt(1, 10);           // bind deptno
            pst.setDouble(2, 50000.0);   // bind salary

            try (ResultSet rs = pst.executeQuery()) {
                while (rs.next()) {
                    System.out.println(rs.getString("ename") +
                        " " + rs.getDouble("salary"));
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Why PreparedStatement: the SQL text is pre-compiled once by the server and re-used for every execution with different bound values, giving (i) better performance for repeated queries, and (ii) protection from SQL-injection because parameter values are sent separately from the SQL text and are never concatenated into the query string.

Q4. Determine conflict-serializability of a 4-transaction schedule using a precedence graph. [5 Marks]

Given Schedule S (four transactions T1, T2, T3, T4 operating on items A, B, C):

```
T1: R(A)          T1: W(A)
T2:      R(A)          W(B)
T3:          W(A)      R(C)
T4:          R(B)          W(C)

Chronological order:
R1(A) R2(A) W3(A) W1(A) R4(B) W2(B) R3(C) W4(C)
```

Step 1 — Identify conflicting operation pairs

Two operations conflict if they belong to different transactions, access the same data item, and at least one is a write. Scanning the schedule left to right for each item:

- On A: R1(A)→W3(A) conflict (T1→T3); R2(A)→W3(A) conflict (T2→T3); W3(A)→W1(A) conflict (T3→T1)
- On B: R4(B)→W2(B) conflict (T4→T2)
- On C: R3(C)→W4(C) conflict (T3→T4)

Step 2 — Draw the precedence (serialization) graph

Nodes = {T1, T2, T3, T4}. Directed edges, one per conflicting pair, drawn from the transaction whose operation occurs earlier to the one occurring later:

```
T1 --> T3
T2 --> T3
T3 --> T1      (back-edge!)
T4 --> T2
T3 --> T4
```

Step 3 — Check for cycles

Following the edges: T1→T3→T1 forms a cycle (T1→T3 and T3→T1 both exist). A precedence graph with a cycle means **no equivalent serial order exists**.

Conclusion: the schedule S is **NOT conflict-serializable** because the precedence graph contains a cycle (T1→T3→T1). If, instead, the W3(A) and W1(A) operations had not conflicted (no back-edge), the graph would have been acyclic, e.g. T4→T2→T3→T4 minus the reverse edge, giving the equivalent serial order T1, T2/T4, T3.

Q5. Demonstrate Basic Two-Phase Locking (2PL); identify and resolve deadlocks. [5 Marks]

Rule of Basic 2PL: every transaction is divided into a *growing phase*, in which it may only acquire (lock) items, and a *shrinking phase*, in which it may only release locks — once the first unlock happens, no further locks may be obtained.

Example transaction set

```
T1: Lock-X(A)  Read(A)  Write(A)  Lock-S(B)  Read(B)  Unlock(A)  Unlock(B)
T2: Lock-X(B)  Read(B)  Write(B)  Lock-S(A)  Read(A)  Unlock(B)  Unlock(A)
```

Step-by-step execution

Time	T1 action	T2 action	Result
t1	Lock-X(A) granted	-	T1 holds A
t2	-	Lock-X(B) granted	T2 holds B
t3	Read/Write A	-	OK
t4	-	Read/Write B	OK
t5	Requests Lock-S(B)	-	BLOCKED (T2 holds X on B)
t6	-	Requests Lock-S(A)	BLOCKED (T1 holds X on A)

Deadlock identification

At t5-t6 both transactions are waiting on each other: T1 waits for B (held by T2), and T2 waits for A (held by T1). Drawing a wait-for graph gives $T1 \rightarrow T2$ and $T2 \rightarrow T1$ — a cycle — confirming a **deadlock**.

Deadlock resolution

- **Detection-based:** the DBMS's deadlock detector periodically builds the wait-for graph, finds the cycle $\{T1, T2\}$, and rolls back one transaction (the *victim*, chosen e.g. by lowest priority or least work done) — say T2. T2's locks are released, its updates undone, and it is restarted.
- **Prevention-based (alternative):** use a timestamp scheme such as Wait-Die/Wound-Wait (see Q6) so that the conflicting lock request is resolved instantly at request time instead of being allowed to wait indefinitely.

After rolling back T2, T1's pending Lock-S(B) request is granted, T1 completes and unlocks A and B (shrinking phase), restoring the system to a serializable, deadlock-free execution.

Q6. Apply Wait-Die and Wound-Wait timestamp-based deadlock-prevention schemes. [5 Marks]

Each transaction T_i is assigned a timestamp $TS(T_i)$ at start (older transaction = smaller timestamp = higher priority). Suppose **$TS(T1)=5$** (older) requests a lock held by **T2** with **$TS(T2)=10$** (younger).

Scheme	Rule when requester is OLDER (TS smaller)	Rule when requester is YOUNGER (TS larger)
Wait-Die (non-preemptive)	Requester WAITS for the lock (older transaction may wait)	Requester DIES (is aborted) and restarts later with its ORIGINAL timestamp
Wound-Wait (preemptive)	Requester WOUNDS the holder (younger holder is aborted/rolled back) and the older requester takes the lock	Requester WAITS for the younger holder to finish

Worked examples

Case A — $T1$ ($TS=5$) requests an item locked by $T2$ ($TS=10$): $T1$ is older than $T2$.

- Wait-Die: since requester $T1$ is older, **$T1$ waits** until $T2$ releases the lock.
- Wound-Wait: since requester $T1$ is older, **$T1$ wounds $T2$** — $T2$ is aborted and its lock released; $T1$ acquires the item.

Case B — $T2$ ($TS=10$) requests an item locked by $T1$ ($TS=5$): $T2$ is younger than $T1$.

- Wait-Die: since requester $T2$ is younger, **$T2$ dies** — it is aborted immediately and restarted later with the same timestamp (5's turn) so it eventually becomes the oldest and succeeds.
- Wound-Wait: since requester $T2$ is younger, **$T2$ waits** for $T1$ to release the lock naturally.

Key property: in both schemes timestamps only ever decrease in the 'wait' direction (a transaction always waits for a younger one, never the reverse), so no cycle can ever form in the wait-for graph — guaranteeing the two schemes are **deadlock-free** by construction, unlike Basic 2PL in Q5.

Q7. Apply the Immediate Update recovery protocol on transaction logs with checkpoints. [5 Marks]

In **Immediate Update**, a transaction's changes may be written to the database *before* it commits, so on crash recovery the log must be scanned to REDO committed transactions and UNDO uncommitted ones.

Given log (with a checkpoint):

----- SYSTEM CRASH -----

Step 1 — Scan backward from crash to the last checkpoint

Determine each transaction's status at crash time: T1 committed (before checkpoint), T2 committed (after checkpoint), T3 has no commit record, T4 has no commit record.

Step 2 — Build REDO and UNDO lists

Transaction	Status at crash	Action
T1	Committed (before checkpoint)	No action needed (already durable before checkpoint)
T2	Committed (after checkpoint)	REDO — reapply <T2,B,200,300> i.e. set B=300
T3	Active / not committed	UNDO — restore C to its before-value 100
T4	Active / not committed	UNDO — restore D to its before-value 400

Step 3 — Apply recovery

- REDO phase (in log order, oldest to newest): re-execute T2's write so B is guaranteed to be 300.
- UNDO phase (in reverse log order, newest to oldest): undo T4's write (D→400) then undo T3's write (C→100), and write <T3,abort> and <T4,abort> to the log.

Result: after recovery, A=700 and B=300 persist (committed, durable), while C=100 and D=400 are restored to their pre-transaction values — the database is brought back to a consistent state reflecting exactly the committed transactions T1 and T2.

Q8. Apply Deferred Update (NO-UNDO/REDO) recovery on a log with committed and uncommitted transactions. [5 Marks]

In **Deferred Update**, all writes of a transaction are held in a private buffer/log and are applied to the actual database only AFTER the transaction commits. Consequently, an uncommitted transaction's changes are never written to disk, so **UNDO is never required** — recovery only ever needs to REDO committed transactions.

Given log file:

---- SYSTEM CRASH (T3 not committed) ----

Step 1 — Classify transactions

Transaction	Commit record present?	Recovery action
T1	Yes	REDO: apply A=700, B=300
T2	Yes	REDO: apply C=150
T3	No	IGNORE — its buffered writes (D=600) were never applied to the DB, so

Step 2 — Apply REDO in the order the writes appear in the log

- Set A = 700
- Set B = 300
- Set C = 150
- T3's write to D is discarded entirely — it was only in the log/buffer, never on disk, so the database is untouched for D.

Conclusion: Deferred Update recovery is a NO-UNDO/REDO algorithm — it is simpler and faster to recover from than Immediate Update because only a REDO pass is ever needed, at the cost of holding all of a transaction's updates in memory/log until commit (reducing concurrency and using more buffer space).

Q9. Design a concurrency-control mechanism using strict 2PL for a banking system (T1 transfers funds, T2 reads balance). [5 Marks]

Scenario: T1 transfers ₹5,000 from Account A to Account B. T2 concurrently reads the balance of Account A for a statement.

T1: Transfer(A->B, 5000)	T2: Read-Balance(A)
Lock-X(A)	Lock-S(A)
Read(A)	Read(A)
A = A - 5000	Print(A)
Write(A)	Unlock(A)
Lock-X(B)	Commit
Read(B)	
B = B + 5000	
Write(B)	
Commit	
Unlock(A) -- held until commit	
Unlock(B) -- held until commit	

Design under Strict 2PL

- **Growing phase:** T1 acquires Lock-X(A) then, after modifying A, Lock-X(B) — all locks acquired before any release, satisfying the 2PL growing-phase rule.
- **Strict rule (the 'strict' part of 2PL):** ALL exclusive locks held by T1 are released only at commit/abort time (not as soon as the shrinking phase could begin). This guarantees that no other transaction can read A or B while T1's transfer is only partially applied.
- **T2's request:** when T2 requests Lock-S(A) while T1 holds Lock-X(A), T2 is forced to BLOCK and wait — it cannot read a half-updated value of A, preventing a dirty read.

- **Recoverability & cascadeless-ness:** because locks are held till commit, if T1 were to abort mid-transfer, T2 could never have seen an uncommitted value of A — so strict 2PL guarantees cascadeless, strict-recoverable schedules, which is essential for money-transfer correctness.

Execution outcome

T2 remains blocked until T1 commits and releases Lock-X(A); T2 then acquires Lock-S(A) and reads the final, consistent post-transfer balance. This design guarantees **serializability** (T1 before T2, or T2 fully before T1 starts — never an interleaving that exposes an inconsistent intermediate balance), which is exactly the correctness property a banking system requires.

Q10. Pseudocode for heuristic query optimization: push down selection, project early, order joins by smallest intermediate result. [5 Marks]

```
ALGORITHM HeuristicOptimize(QueryTree T):

1. // Rule 1: Break conjunctive selections
   FOR EACH selection node sigma(c1 AND c2 AND ... AND cn) in T:
       replace it with a chain sigma(c1) -> sigma(c2) -> ... -> sigma(cn)

2. // Rule 2: Push SELECT operations down
   FOR EACH selection node sigma(cond) in T:
       WHILE sigma can be moved further down the tree
           (i.e. all attributes in 'cond' belong to the
            child sub-tree's relations):
           move sigma below its current parent,
           as close as possible to the leaf relation it restricts

3. // Rule 3: Push PROJECT operations down / insert early projections
   FOR EACH leaf relation R in T:
       determine attrib_needed(R) = attributes of R required by
           later operators (joins, selects, final output)
       insert PROJECT(attrib_needed(R)) directly above R

4. // Rule 4: Combine CARTESIAN PRODUCT + SELECT into JOIN
   FOR EACH pair (CROSS_PRODUCT(R,S), sigma(R.a = S.b)) in T:
       replace with JOIN(R, S ON R.a = S.b)

5. // Rule 5: Order joins by increasing estimated result size
   join_list = all JOIN nodes remaining in T
   FOR EACH join J in join_list:
       estimate |J| using selectivity/statistics
           (e.g. |R Join S| ≈= |R| * |S| / max(V(a,R), V(b,S)))
   SORT join_list by estimated |J| ascending
   rebuild left-deep join tree so that the join producing the
       SMALLEST intermediate result is executed FIRST,
       then successively join in the next-smallest-result relation

6. RETURN optimized tree T
```

Explanation: steps 1-2 minimise the number of tuples flowing into later operators as early as possible (reduce cardinality); step 3 minimises the number of columns carried forward (reduce tuple width); step 4 avoids ever materialising a full Cartesian product; and step 5 ensures that at every stage the smallest possible intermediate relation is produced, which keeps the input size to every subsequent join as small as possible — directly minimising overall I/O and CPU cost, which is the goal of cost-aware heuristic

optimization.

--- End of Answer Script ---